

Week 8

Questions

**

- Answer the following questions in a section called “Q & A”:
- Each bullet point can be between 1-5 sentences. You choose the amount of detail as long as I see that you understand it.
 - What is the difference between high availability and fault tolerance? Which is best to strive for?
 - High Availability focuses on minimizing service and app downtime/disruption. Goal is minimal downtime.
 - Load Balancers, Health Checks, Multi-Zone
 - Fault tolerance focuses on maintaining service availability during failure and recovery. Goal is zero downtime.
 - Auto-scaling w/ load balancer, Multi-Region
 - Fault Tolerance is best to strive for if budget is available. Fault Tolerant design is more expensive than just High Availability.
 -
 - Explain the difference between autoscaling and elasticity. What is vertical and horizontal autoscaling? Is one better? Are they feasible on prem?
 - Autoscaling is the ability to scale the number of VMs up or down based on incoming workload.
 - Elasticity is automation for dynamic vertical and/or horizontal scaling based on incoming workload.
 - Vertical Autoscaling is scaling up in terms of increasing a servers capacity (increasing CPU/Memory/Storage)
 - Horizontal Autoscaling is scaling out in terms of adding more servers to sharing the workload (adding more VMs to the MIG)
 - Horizontal is better than vertical since it improves availability and fault tolerance.
 - Not feasible on prem, in terms of expanding past current capacity, since you need to scale up and down based on workload which would be difficult for physical servers.
 - Explain what the difference between managed and unmanaged instance groups is.
 - Managed Instance Groups: This is for managing multiple identical VMs in terms of scalability & high availability utilizing automated services like autoscaling or autohealing. Recommended by Google.
 - Unmanaged Instance Groups: This is for user-managing load balancing across non-

identical VMs. Doesn't include existing automated configuration for autoscaling or autohealing.

- Explain the different use cases for health checks used by applications (in instance groups) and health checks used by load balancers. Can they be the same? Are they different API calls? Should they be the same?
 - MIG Health Checks are used to check a VMs health and signal the need to delete and recreate instances
 - Load Balancing Health Checks are used to direct traffic away from unhealthy VMs to healthy VMs
 - Technically they can be the same but it is recommended they are different since the health check for a load balancer should be more aggressive because these checks route user traffic. The health checks for MIGs should be less aggressive since they are checking if autohealing is needed. Both have the same API calls.
- Explain in a few sentences what the 3 tier architecture is and how it relates to what you are learning.
 - 3 Tier architecture is made up of 3 design levels
 - Web Tier - Application frontend that usually receives traffic from an external ALB to send the traffic to the frontend MIGs which will then send to the internal ALB
 - Application Tier - This is the internal MIGs that receive traffic from the internal ALB
 - Database Tier - This level has the databases that receive traffic from the internal MIGs
 - Relates to what we are learning since we are doing MIGs and Load balancers.

**

Runbook

Goal of runbook

- The goal of this runbook is to provide instruction on how to create and manage an instance group (MIG) in GCP.
- This runbook will walk through the clickops process of setting this up with the proper configuration and autoscaling/autohealing.
- This runbook is for stateless multi-zone MIG.

Prerequisites

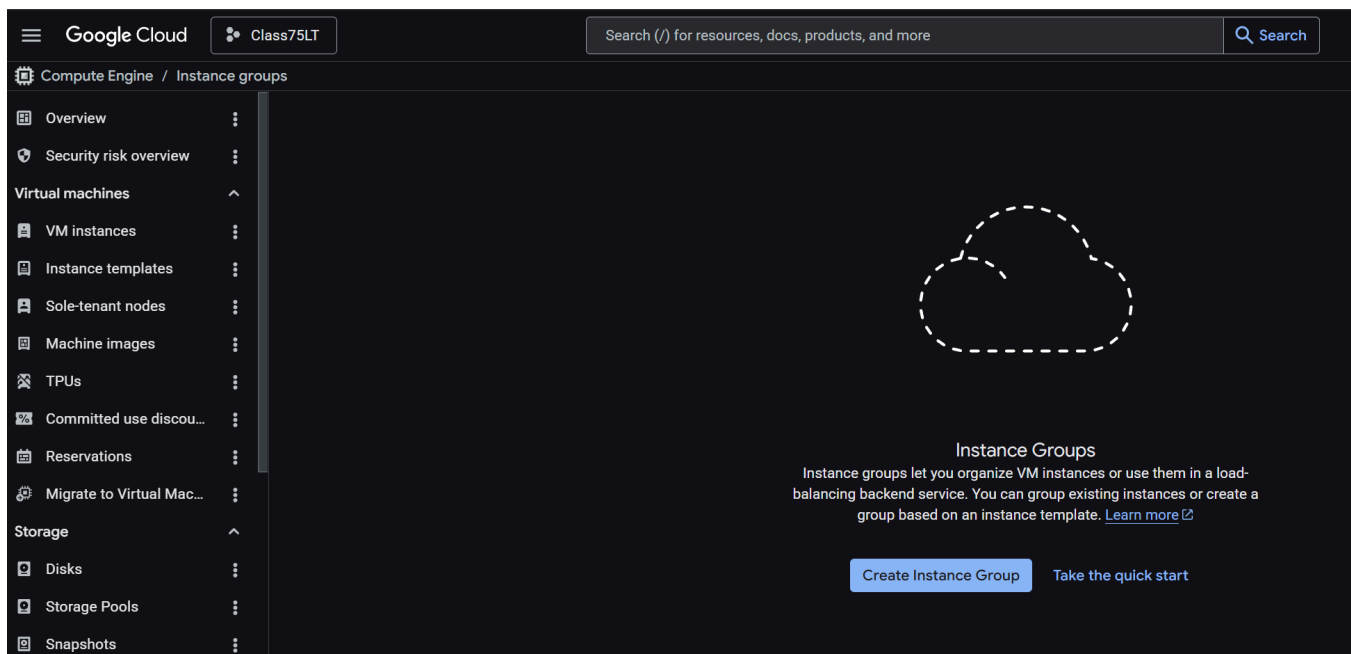
GCP Account

Instance Template

Process Steps

Create MIG

- In GCP navigate to Instance Groups and click create Instance Group



MIG Name & Type

- Select Stateless MIG
- Give MIG a name, description, instance template, desired number of instances

Navigation menu (.) / Instance groups / Create instance group

Create Instance Group

New managed instance group (stateless)
Automatically manage groups of VMs that do stateless serving and batch processing.

New managed instance group (stateful)
Automatically manage groups of VMs that have persistent data or configurations (such as databases or legacy applications).

New unmanaged instance group
Manually manage groups of load balancing VMs.

Set up automatic management for a group of stateless VMs, including updates, regional deployments, load balancing, autoscaling, and autohealing. [Learn more](#)

Name *
runbook-stateless-mig
Name is permanent

Description
Stateless MIG that is multi-zonal

Instance template *
supera
e2-medium, debian-12-bookworm-v20260428, us-central1

Number of instances *
6

☐ Use resize request to create VMs all at once
Create and run VMs all at once when resources become available. Use as an alternative to VMs getting created on an individual basis. [Learn more](#)

Location

- In location, select Multi-zones, select region and zones
- Leave Target distribution as Balanced

Autohealing

- Add a Health check
- Scope - Regional
- Select region, protocol & port
- Turn on logs
- Set Health Criteria

Google Cloud | Class7SLT | Search (/) for resources, docs, products, and more

Compute Engine / Instance groups / Create instance group

Create Instance Group

VM instance lifecycle

Configure what happens when VMs are created, repaired, and deleted.

Action on failure

Select the action to take when a VM in your instance group fails. [Learn more](#)

Default action on failure: **Repair instance**

Autohealing

Autohealing recreates VM instances if your application cannot be reached by the health check. [Learn more](#)

Health check: **runbook-hc**

Filter Type to filter

- No health check
- runbook-hc (TCP)
- mig1-hc (TCP)
- port: 80, timeout: 5s, check interval: 5s, unhealthy threshold: 2 attempts, location: us-central1
- mig2-hc (TCP)
- port: 80, timeout: 300s, check interval: 300s, unhealthy threshold: 2 attempts, location: us-central1

[Edit runbook-hc](#)

Updates during VM instance repair

☐ Keep the same instance configuration
Recreate the VM with the same instance template and per-instance configuration that was used to create it

☒ Update instance configuration
Apply the latest instance template and per-instance configuration when recreating the VM

Change zone during VM instance repair

☒ Allow repair of VMs in an alternate zone
If instance cannot be repaired in the original zone, it will be repaired in another zone of the

Health Check

Name * **runbook-hc**

Description **Health check for runbook**

Scope ☐ Global ☒ Regional

Region **us-central1 (Iowa)**

Protocol **TCP** Port * **80**

PROXY protocol **None**

Request Response

Logs

☒ On
Turning on Health check logs can increase costs in Logging.

☐ Off

Health criteria

Define how health is determined: how often to check, how long to wait for a response, and how many successful or failed attempts are decisive

Check interval * **10** seconds Timeout * **10** seconds

Healthy threshold * **1** consecutive successes

Unhealthy threshold * **3** consecutive failures

Create **Cancel** **Equivalent code** **Save** **Cancel**

Click create to create the MIG

Verification MIG is working

Navigate to the Instance Group and click on the MIG

Review the Overview section to see the running instances

Google Cloud | Class7SLT | Search (/) for resources, docs, products, and more

Compute Engine / Instance groups / Instance group: runbook-stateless-mig

runbook-stateless-mig **Edit** **Restart/replace VMs** **Delete Group** **Equivalent code**

Overview Details Monitoring Errors

Instances by status

6 instances **100% healthy**

Autoscaling **On (min 6, max 10)**

Based on 1 metric and 0 schedules

Status **Ready**

Creation Time **May 7, 2026, 10:05:50 PM UTC-05:00**

Description

Target running size **6**

Template **supera (Regional)**

Location **us-central1 (3/4)**

Resize requests **None**

VM instances **Suspend** **Stop** **Start / Resume** **Remove from group** **Delete**

Filter Enter property name or value

Status	Name	Creation Time	Template	Zone	Per instance config	Internal IP	External IP	Health Check Status	Connect
☒	runbook-stateless-mig-0zk5	May 7, 2026, 10:06:05 PM UTC-05:00	supera (Regional)	us-central1-c		10.128.0.15 (nic0)	34.30.219.49	Healthy	SSH
☒	runbook-stateless-mig-41q5	May 7, 2026, 10:06:05 PM UTC-05:00	supera (Regional)	us-central1-b		10.128.0.14 (nic0)	34.44.135.245	Healthy	SSH
☒	runbook-stateless-mig-5gzz	May 7, 2026, 10:06:06 PM UTC-05:00	supera (Regional)	us-central1-f		10.128.0.17 (nic0)	34.69.18.127	Healthy	SSH
☒	runbook-stateless-mig-cxc9	May 7, 2026, 10:06:06 PM UTC-05:00	supera (Regional)	us-central1-f		10.128.0.19 (nic0)	35.202.150.176	Healthy	SSH
☒	runbook-stateless-mig-dqps	May 7, 2026, 10:06:06 PM UTC-05:00	supera (Regional)	us-central1-c		10.128.0.18 (nic0)	104.154.165.74	Healthy	SSH
☒	runbook-stateless-mig-zt7f	May 7, 2026, 10:06:06 PM UTC-05:00	supera (Regional)	us-central1-b		10.128.0.16 (nic0)	136.115.45.246	Healthy	SSH

Terraform VM Breakdown

Required arguments for VM instance (google_compute_instance)

name, boot_disk, machine_type

Output internal and external IP addresses of provisioned VM

How I figured this out

The vm terraform resource registry page, the attributes section shows what can be exported (used for terraform outputs) -

https://registry.terraform.io/providers/hashicorp/google/latest/docs/resources/compute_instance#attributes-reference

The gcp output hashicorp documentation -

<https://developer.hashicorp.com/terraform/tutorials/gcp-get-started/google-cloud-platform-outputs>

Outputs

```
terraform output "internal ip" { value =  
google_compute_instance.vm_instance.network_interface.0.network_ip }  
terraform output "external ip" { value =  
google_compute_instance.vm_instance.network_interface.0.access_config.0.nat_ip }
```

2 Non-required VM arguments

...

...

...

...

Additional Resources Used

<https://www.nobl9.com/service-availability/high-availability-vs-fault-tolerance>

<https://www.scalecomputing.com/resources/fault-tolerance-vs-high-availability>

<https://aerospike.com/blog/understanding-elasticity-scalability-cloud-computing/>